

PRÁTICA DE ARDUINO:

FUNDAMENTOS E PRIMEIROS PASSOS

1. Estrutura Fundamental do Código em C/C++

Um programa em Arduino (sketch) é escrito em C/C++ e segue uma estrutura fixa para interagir com o hardware.

#define e Constantes de Pinagem

Antes do programa iniciar, definimos nomes para os pinos. A diretiva #define substitui todas as ocorrências do nome pelo valor numérico antes de compilar o código, sem consumir memória RAM.

C++

```
#define LED_PIN 8 // Substitui 'LED_PIN' pelo número 8 na compilação
```

void setup()

Função de **configuração inicial**, executada **apenas uma vez** ao ligar ou reiniciar a placa. Serve para configurar pinos (entrada/saída) e iniciar comunicações.

void loop()

Função principal executada em **ciclo infinito**. Assim que o setup() termina, o loop() roda continuamente linha por linha até a placa ser desligada.

Comentários

Anotações ignoradas pelo compilador, usadas para documentar o código.

- **Linha única (//):** Comenta apenas o que está à direita na mesma linha.
- **Múltiplas linhas (/* */):** Comenta blocos inteiros de texto.

2. Primeiras Funções Nativas

- **pinMode(pino, modo):** Configura o comportamento elétrico de um pino. OUTPUT prepara o pino para fornecer energia (saída). INPUT prepara para ler sinais (entrada).
- **digitalWrite(pino, estado):** Envia um sinal digital para uma saída. HIGH aplica 5V (liga), LOW aplica 0V (desliga).
- **delay(milissegundos):** Pausa a execução do programa pelo tempo especificado (1000 ms = 1 segundo).

3. Projeto Prático 1: Piscar LED (Blink)

O Código

C++

```
#define LED_EXTERNO 8
#define LED_INTERNO 13

void setup() {
  pinMode(LED_EXTERNO, OUTPUT);
  pinMode(LED_INTERNO, OUTPUT);
}

void loop() {
  // LIGAR
  digitalWrite(LED_EXTERNO, HIGH);
  digitalWrite(LED_INTERNO, HIGH);
  delay(1000); // Aguarda 1 segundo

  // DESLIGAR
  digitalWrite(LED_EXTERNO, LOW);
  digitalWrite(LED_INTERNO, LOW);
  delay(1000); // Aguarda 1 segundo
}
```

Transição de Estados

O código gera uma onda quadrada de tensão:

- 1. **5V:** digitalWrite em HIGH. LED acende.
- 2. **Pausa:** delay(1000) congela em 5V por 1 segundo.
- 3. **0V:** digitalWrite em LOW. LED apaga.
- 4. **Pausa:** delay(1000) congela em 0V por 1 segundo, e o ciclo reinicia.

Montagem Física

Componente	Terminal	Conectado em
LED	Anodo (+)	Pino Digital 8 do Arduino
LED	Catodo (-)	Terminal A do Resistor de 220 Ω
Resistor	Terminal B	Pino GND do Arduino

Segurança: Nunca ligue um LED diretamente aos pinos sem um resistor em série para limitar a corrente.

4. Simulação no Tinkercad

O Tinkercad (tinkercad.com) permite simular o circuito sem peças físicas:

1. **Crie o projeto:** Faça login, vá em "Projetos" > "Criar" > "Circuito".
2. **Adicione as peças:** Arraste um Arduino Uno, Protoboard, LED e Resistor (altere para 220 Ω) para a tela.
3. **Faça a fiação:** Ligue uma perna do resistor ao Catodo do LED e ao pino **GND**. Ligue o Anodo do LED ao **Pino 8**.
4. **Insira o código:** Clique em **Código** (</>), mude de "Blocos" para "Texto" e cole o código C++.
5. **Teste:** Clique em **Iniciar simulação** para ver o LED piscar.

5. Projeto 2: Entradas Digitais e Botões

Para ler ações do usuário, usamos o `digitalRead(pino)`. Ele retorna HIGH se houver energia, ou LOW se não houver.

Montagem do Botão (Pull-Down)

Um resistor de 10 k Ω é necessário para garantir que o pino leia 0V (LOW) quando o botão estiver solto.

1. Ligue um lado do botão ao **5V**.
2. Ligue o outro lado ao **Pino Digital 2**.
3. Na mesma perna ligada ao Pino 2, conecte o resistor de 10 k Ω indo para o **GND**.

Código: Ligar enquanto pressionado

C++

```
#define LED_EXTERNO 8
#define BOTAO_PIN 2

void setup() {
  pinMode(LED_EXTERNO, OUTPUT);
  pinMode(BOTAO_PIN, INPUT); // Configurado como entrada
}

void loop() {
  int estadoBotao = digitalRead(BOTAO_PIN);

  if (estadoBotao == HIGH) {
    digitalWrite(LED_EXTERNO, HIGH); // Liga se pressionado
  } else {
```

```
    digitalWrite(LED_EXTERNO, LOW); // Desliga se solto
}
}
```

6. Projeto 3: Botão como Interruptor (Toggle e Debounce)

Botões físicos sofrem de *bouncing* (os contatos metálicos vibram milissegundos antes de estabilizar, gerando leituras falsas). Usamos *debounce* (um pequeno delay) para ignorar essa vibração, e variáveis de memória para fazer o botão funcionar como um interruptor liga/desliga.

Código: Toggle

C++

```
#define LED_EXTERNO 8
#define BOTAO_PIN 2

int estadoLED = LOW;
int estadoBotaoAnterior = LOW;

void setup() {
    pinMode(LED_EXTERNO, OUTPUT);
    pinMode(BOTAO_PIN, INPUT);
}

void loop() {
    int estadoBotaoAtual = digitalRead(BOTAO_PIN);

    // Detecta a transição exata do clique
    if (estadoBotaoAtual == HIGH && estadoBotaoAnterior == LOW) {

        // Inverte a memória do LED
        if (estadoLED == LOW) {
            estadoLED = HIGH;
        } else {
            estadoLED = LOW;
        }

        digitalWrite(LED_EXTERNO, estadoLED);

        // DEBOUNCE: Congela 200ms para ignorar o "quique" mecânico
        delay(200);
    }

    // Atualiza a memória para o próximo ciclo
    estadoBotaoAnterior = estadoBotaoAtual;
}
```